

CS213 2025 Tutorial Solutions

CS213/293 UG TAs

2025

Contents

Tutorial 4 [2](#)

Tutorial 4

1. Given a tree with a maximum number of children as k . We give a label between 0 and $k - 1$ to each node with the following simple rules.

(i) The root is labelled 0.

(ii) For any vertex v , suppose that it has r children, then arbitrarily label the children as $0, \dots, r - 1$.

This completes the labelling. For such a labelled tree T , and a vertex v , let $Seq(v)$ be the labels of the vertices of the path from the root to v . Let $Seq(T) = \{Seq(v) \mid v \in T\}$ be the set of label sequences.

What properties does $Seq(T)$ have? If a word w appears, what words are guaranteed to appear in $Seq(T)$? How many times does a word w appear as a prefix of some words in $Seq(T)$?

Solution: Some properties are $Seq(T)$ are:-

1. Size of this set will be n (n is the number of nodes in the tree) (See that for different v_1, v_2 vertices, $Seq(v_1) \neq Seq(v_2)$).
2. If there are h_j nodes of j length in set $Seq(T)$, then the tree has h_j nodes at j^{th} level.
3. If we consider ordering of set $Seq(T)$ in lexicographical order as the metric, then it will just represent a DFS on the tree.
4. If we consider ordering of set $Seq(T)$ by length followed by lexicographical order as the metric, then it will just represent a BFS on the tree.

$p_0 = 0$ (root of tree)

$$w = p_0 p_1 p_2 \dots p_n$$

$$S_w = \{x \mid x = p_0 p_1 p_2 \dots p_i q \text{ where } 0 \leq i \leq n - 1 \text{ \& } 0 \leq q \leq p_{i+1}\} \cup \{0\}$$

We can prove S_w is the set which is guaranteed to be present if w is present by following argument:- Each element of set S_w is guaranteed to be present if w is present. We can prove this by considering two claims,

1. Any prefix of w will be present since this just represents the path from root to ancestor of w .
2. If a word $l_0 l_1 l_2 \dots l_n$ is present then all the words of form $l_0 l_1 l_2 \dots l_{n-1} q$ where $0 \leq q \leq l_n$ will also be present since these are just the child of the node represented by sequence $l_0 l_1 l_2 \dots l_{n-1}$

If $w = Seq(v)$ for some $v \in T$ and the number of nodes in the subtree of v be x , then w appears as a prefix for x number of words in $Seq(T)$.

2. Write a function that returns $lca(v, w, T)$. What is the time complexity of the program?

Solution: This is for a general tree and not just a binary tree.

See the Pseudo-code in Algorithm 1.

Firstly, we calculate $depth[\cdot]$ and $parent[\cdot]$ of each node using dfs in the tree until we assign them for both v, w . Now, if the depth of both the nodes is same, we start moving

up to their parent pointers until it is the same for both. Else if both are not on the same depth, we bring the deeper node to equal depth by hopping to its parent node, and then we go back to the algorithm for same depth.

Algorithm 1: LCA

```

1 Function LCA( $v, w$ ):
2   while depth[ $v$ ] > depth[ $w$ ] do
3     |  $v \leftarrow$  parent[ $v$ ];
4   end
5   while depth[ $w$ ] > depth[ $v$ ] do
6     |  $w \leftarrow$  parent[ $w$ ];
7   end
8   while  $v \neq w$  do
9     |  $v \leftarrow$  parent[ $v$ ];
10    |  $w \leftarrow$  parent[ $w$ ];
11  end
12  return  $v$ ;
13 end

```

3. Given $n \in T$, Let $f(n)$ be a vector, where $f(n)[i]$ is the number of nodes at depth i from n .
- Give a recursive equation for $f(n)$.
 - Give a pseudo-code to compute the vector $f(\text{root}(T))$. What is the time complexity of the program?

Solution:

Observations:

- For any node n , the only node at depth 0 from n is the node n itself.
- For a leaf node l , there is no node n such that $\text{depth}(n)$ from l is greater than or equal to 1.
- For a node n , if node m is at depth k , then there exists a *unique path* from n to m of length k , and this path must go through one of the children of n (let's say p) i.e. node m is at a depth $k - 1$ from node p .
Conversely, if a node m is at a depth $k - 1$ from node p , then it must be at a depth of k from node n where n is parent of p .

With these observations in mind, we can say that $f(n)[i] = \sum_{m \in \text{children}(n)} f(m)[i - 1]$.
Recursive equation of $f(n)$ then can be written as:

$$f(n) = [1] + \left(\sum_{m \in \text{children}(n)} f(m) \right)$$

which is nothing but concatenating $[1]$ in front of the sum obtained from the f vectors of the children.

Here base case will be when n is a leaf, for that we have

$$f(n) = [1, 0, 0, \dots, 0]$$

Pseudocode: (Look at Algorithm 2)

As you all know, we will be doing some kind of traversal to get $f(\text{root}(T))$

Can you think of the traversal method we gonna use by looking at the recursion obtained in **part 1** ???

I hope you got it correct, we are going to use post-order traversal (why so?)

We will use array data structure for storing $f(i)$ of size n , where n is the number of nodes in the tree

The algorithm given above, does a post order traversal on tree (note that this algorithm holds good for any tree and not just a binary tree), on getting to a leaf, it returns an array $[1, 0, \dots, 0]$, and on an internal node, it returns f according to the recursion obtained above.

Time complexity

Assume that vectors at all the nodes are of size n . There will be $\mathcal{O}(n)$ edges and for each edge we will do addition of two such vectors. Hence this algorithm has $\mathcal{O}(n^2)$ time complexity.

Algorithm 2: Computing $f(n)$

Data: Tree T having n nodes

```
1 Function ModifiedPostOrderWalk( $p$ ):  
2    $f \leftarrow [1, 0, \dots, 0]$   
3   if  $p.isLeaf()$  then  
4     return  $f$   
5   end  
6   for  $n' \in children[p]$  do  
7      $f_c \leftarrow ModifiedPostOrderWalk(n')$   
8     for  $j \leftarrow 1$  to  $n - 1$  do  
9        $f[j] \leftarrow f[j] + f_c[j - 1]$   
10    end  
11  end  
12  return  $f$   
13 end
```

4. Give an algorithm for reconstructing a binary tree if we have the preorder and inorder walks.

Solution: The key idea here is to note that we will recursively use both traversals in parallel. We will use the inorder traversal for finding the node index and iterate over the preorder to build the tree.

We assume that we have a function that can search for a node in the inorder traversal given the start and end indices of the search range.

To utilise both the traversals, we maintain a start and end index for the inorder traversal of the tree and make recursive calls while moving these indices and the iterator of the preorder traversal.

The tree build function can be written recursively as follows:

- if the start index is greater than the end index the tree is *NULL*
- otherwise we first create the root as the first element of the preorder
- if the start index and the end index are equal then the root is the answer
- otherwise we search for the index of the root in the inorder traversal

- the left sub-tree can be built recursively by setting the end index as the index previous to the root while the right sub-tree can be built after this by setting the start index as the index next to the root (note that the preorder traversal will be linearly iterated over by both these recursive calls but the iterator will be common across the calls)
- we return the root after setting the results of the sub-tree computations

Explanation: Note that if we know the position of the root in the inorder traversal, the left side of the inorder traversal is the inorder traversal of the left sub-tree and similarly for the right sub-tree. We also note that the first element in the preorder traversal is always the root, and the following elements will be the preorder traversal of the left sub-tree followed by the right sub-tree. The time complexity of this will be $\mathcal{O}(n^2)$ (think what is the worst case).

Optimisation: Instead of linearly searching for the index of a node in the in-order traversals, we store them in a hashmap, giving $\mathcal{O}(1)$ search. Time complexity of this algorithm will be $\mathcal{O}(n)$

Correctness: Arguing for correctness is straightforward. Within each recursive call, the root is the element pointed to by the preorder iterator, which increments by one for each recursive call. After that, we find the root in the inorder traversal and then build the tree using the left side of the inorder traversal and the next set of the elements in the preorder traversal (both of these correspond to the left sub-tree). Then we build the tree using the right side of the inorder traversal and the remaining set of the elements in the preorder traversal (both correspond to the right sub-tree). Since the invariants (left-root-right for inorder and root-left-right for preorder) are maintained, the recursion terminates correctly.

Additional: What is the **minimal** set of changes required for building a tree using the postorder and inorder traversals?

Algorithm 3: Reconstruct binary tree from preorder[1..n] and inorder[1..n]

Input: preorder[1..n], inorder[1..n]

Output: root of reconstructed binary tree

```

// create hashmap pos such that pos[value] = index in inorder
1 for i ← 1 to n do
2   | pos[inorder[i]] ← i;
3 end
4 Function BuildTree(preL, preR, inL, inR):
5   | if preL > preR or inL > inR then
6   |   return null;
7   end
8   rootVal ← preorder[preL];
9   rootIndex ← pos[rootVal];
10  ; // index of root in inorder
11  leftSize ← rootIndex - inL;
12  root ← new Node(rootVal);
13  root->left ← BuildTree(preL + 1, preL + leftSize, inL, rootIndex - 1);
14  root->right ← BuildTree(preL + leftSize + 1, preR, rootIndex + 1, inR);
15  return root;

// initial call
15 return BuildTree(1, n, 1, n);

```

5. Let us suppose all internal nodes of a binary tree have two children. Give an algorithm for reconstructing the binary tree if we have the preorder and postorder walks.

Solution:

Let preorder walk be $f = (f_1, f_2, \dots, f_n)$ and postorder walk be $l = (l_1, l_2, \dots, l_n)$

Base case

f and l have just 1 element let's say e , then the tree is nothing but a single-node tree with root as e .

Recursion step

Observations for tree with each internal node having two children and number of nodes greater than 1

1. In preorder walk, *first* element is *root* and *second* element is left child of root.
2. In postorder walk, *last* element is *root* and *second last* element is right child of root.

So, now with these observations in mind let's try to build the tree from f and l .

We have the root as $r = f_1 = l_n$

Left child of root as f_2 and right child of root as l_{n-1}

Now we will break f and l so as to get preorder and postorder walks for left and right subtree

Given f , which element in right subtree of r is first to be in the f ?

By Observation 1, preorder on right subtree (which is a tree) of r should have its root i.e. right child of r which is l_{n-1} first in the walk and so we have the break of f !!!

Find l_{n-1} in f , let us say f_k , then we have $f' = (f_2, f_3, \dots, f_{k-1})$ as preorder walk of left subtree of r and $f'' = (f_k, f_{k+1}, \dots, f_n)$ as preorder walk of right subtree of r

Similarly, given l , which element in left subtree of r is the last to be in the l ?

By Observation 2, postorder on left subtree (which is a tree) of r should have its root i.e. left child of r which is f_2 last in the walk and so we have the break of l !!!

Find f_2 in l , let us say l_k , then we have $l' = (l_1, l_2, \dots, l_k)$ as postorder walk of left subtree of r and $l'' = (l_{k+1}, l_{k+2}, \dots, l_{n-1})$ as postorder walk of right subtree of r

With f' and l' , we **recursively** construct the left subtree and with f'' and l'' , the right subtree. The time complexity will again be $\mathcal{O}(n^2)$.

Optimisation: Each node is processed once; if we use a hashmap to locate indices in postorder, total time is $\mathcal{O}(n)$.

Algorithm 4: Reconstruct binary tree from preorder and postorder (full binary tree)

Input: preorder[1..n], postorder[1..n]

Output: root of reconstructed tree

```
1 Function BuildTree(preL, preR, postL, postR):
2   if preL > preR then
3     |   return null;
4   end
5   rootVal  $\leftarrow$  preorder[preL];
6   root  $\leftarrow$  new Node(rootVal);
7   if preL = preR then
8     |   return root;
9   end
10  leftVal  $\leftarrow$  preorder[preL + 1];
11  idx  $\leftarrow$  position of leftVal in postorder[postL..postR] ;
12  leftSize  $\leftarrow$  idx - postL + 1;
13  root.left  $\leftarrow$  BuildTree(preL + 1, preL + leftSize, postL, idx);
14  root.right  $\leftarrow$  BuildTree(preL + leftSize + 1, preR, idx + 1, postR - 1);
15  return root;

// initial call
16 return BuildTree(1, n, 1, n);
```

6. (a) Show that in order printing of BST nodes produces a sorted sequence of keys.
(b) Give a sorting procedure using BST.
(c) Give the complexity of the procedure.

Solution:

- (a) At any non-leaf BST node, the key at any node in the left subtree is not greater than the key at the node. Similarly, the key at any node in the right subtree is not less than the key at the node. An inorder traversal of a BST visits the left subtree first, then the node, and then the right subtree. Thus, the inorder traversal of the BST will produce a sorted sequence of keys.
- (b)
- Create a BST with the first element of the unsorted array as the root and no children.
 - Insert each of the remaining elements into the BST.
 - Perform an inorder traversal of the BST to get the sorted sequence of keys.
- (c) The complexity of the procedure is $\mathcal{O}(n \log n)$ on average. This is because average insertion of n elements in a BST takes $\mathcal{O}(n \log n)$ time. And the inorder traversal takes $\mathcal{O}(n)$ time. In the worst case, when the array is already sorted ascending or descending, each insertion can take $\mathcal{O}(n)$ time, giving an overall complexity of $\mathcal{O}(n^2)$. This is because the binary tree in this case is unbalanced.

7. Given a BST T and a key k , the task is to delete all keys less than k from T . Please write a pseudocode for this. What is the running time of your algorithm? What is the structure of the leftover tree? What is the root of the tree?

Algorithm 5: Deleting all keys less than k in BST

Data: BST T having n nodes, key k

```
1 Function DeleteLessThan( $node, k$ ):  
2   if  $node = NULL$  then  
3     return  $NULL$ ;  
4   end  
5   if  $node.key < k$  then  
6     // discard the node and its left child  
7     return DeleteLessThan( $node.right, k$ );  
8   end  
9   else  
10    // check the left child  
11     $node.left \leftarrow$  DeleteLessThan( $node.left, k$ );  
12    return  $node$ ;  
13  end  
Result:  $T.root \leftarrow$  DeleteLessThan( $T.root, k$ )
```

Solution:**Correctness**

- If $node.key < k$ then every key in $node.left$ is $< node.key < k$, so the whole left subtree (and the node) must be removed. **return** DeleteLessThan($node.right, k$).
- If $node.key \geq k$ then $node$ should be kept. All keys in the right subtree are $\geq node.key \geq k$, only the left subtree may contain keys $< k$, so we recursively prune the left subtree.

Time complexity

At each node, either we move to its right child, or its left child, never both. Hence the time complexity of the removal is $\mathcal{O}(h)$, where h is the height of the BST. For a Balanced BST, we get the best case complexity of $\mathcal{O}(\log n)$, whereas the worst case will be $\mathcal{O}(n)$.

Structure of leftover tree

The leftover tree will remain a BST, this is because if the left child $v- > l$ of a node v is to be removed, it is either replaced by null, or its right child $n- > l- > r$, which has elements smaller than that at the node n , hence BST property is preserved

Root of the tree

- $root.key \geq k$: The root of the tree remains unchanged
- $root.key < k$: then the algorithm replaces the root by recursively returning DeleteLessThan($root.right, k$), i.e., the first node reached by repeatedly following right-child pointers from the original root until a node with key $\geq k$ is found. If there is no such node, then $NULL$ is returned as the root, indicating empty tree.

8. Let $H(n)$ be the expected height of the tree obtained by inserting a random permutation of $1, \dots, n$. Write the recurrence relation for $H(n)$.

Solution: The height of a binary tree equals one plus maximum of that of left and right subtrees. In a random permutation of $[n]$, any element (say k) can be the root of the subtree with probability $1/n$. The left subtree will contain $k - 1$ nodes and the right one $n - k$.

Let X_n be the random variable representing the height of a n -node BST. Also, $H(n) = E[X_n]$. Then the recurrence relation is given as :-

$$\begin{aligned}\forall n \in \mathbb{N}, H(n) &= E[X_n] = 1 + \frac{1}{n} \left(\sum_{i=1}^n E[\max(X_{i-1}, X_{n-i})] \right) \\ &\leq 1 + \frac{1}{n} \left(\sum_{i=1}^n E[X_{i-1} + X_{n-i}] \right) \\ &= 1 + \frac{1}{n} \left(\sum_{i=1}^n E[X_{i-1}] + \sum_{i=1}^n E[X_{n-i}] \right) \\ &= 1 + \frac{2}{n} \left(\sum_{i=1}^{n-1} E[X_i] \right) = 1 + \frac{2}{n} \left(\sum_{i=1}^{n-1} H_i \right)\end{aligned}$$

Base Case: $H(0) = 0$

Solving the recurrence gives $H(n)$ as $\mathcal{O}(\log n)$. You can refer [this](#) for a rough idea of its solution.

9. Given a BST tree T and a value v , write a program to locate the leftmost and rightmost occurrence of the value v .

Solution: Leftmost occurrence in a BST means leftmost occurrence in the inorder traversal of that BST.

Look at Algorithm 6 for the Pseudo-code of the leftmost occurrence in a BST.

The code itself is self-explanatory. Similarly you can write this for the rightmost occurrence.

Time complexity is $\mathcal{O}(h)$, h being the height of T , as we go through each level atmost once.

Algorithm 6: Finding the leftmost occurrence

Data: Tree T having n nodes

```
1 Function Leftmost( $curr, v$ ):
2   if  $curr = NULL$  then
3     return  $NULL$ 
4   end
5   if  $curr \rightarrow val \geq v$  then
6      $left\_check \leftarrow Leftmost(curr \rightarrow left, v)$ 
7     if  $left\_check \neq NULL$  then
8       return  $left\_check$ 
9     end
10  end
11  if  $curr \rightarrow val = v$  then
12    return  $curr$ 
13  end
14  if  $curr \rightarrow val \leq v$  then
15     $right\_check \leftarrow Leftmost(curr \rightarrow right, v)$ 
16    if  $right\_check \neq NULL$  then
17      return  $right\_check$ 
18    end
19  end
20  return  $NULL$ 
21 end
22 // Call  $Leftmost(root, v)$  in the main function
```
